

Experiment 09

Stored Procedures in PostgreSQL

Name: Lakshdeep
UID: 24BCF10059

Aim

To study and implement PostgreSQL stored procedures using IN/OUT parameters, conditional (IF... ELSE) logic, RAISE NOTICE messages, and multi-table INSERT/UPDATE operations to automate real-world business transactions.

Question 1: Gender-wise Employee Count (Medium)

An organization stores employee details in the employee_info table. Create a PostgreSQL stored procedure that accepts an employee's gender as an input parameter and calculates the total number of employees belonging to that gender. The calculated count should be returned through an OUT parameter and should also be displayed as a message using RAISE NOTICE.

Answer

```
CREATE OR REPLACE PROCEDURE pr_count_by_gender(  
    IN p_gender VARCHAR,  
    OUT p_count INT  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    SELECT COUNT(*)  
    INTO p_count  
    FROM employee_info  
    WHERE gender = p_gender;  
  
    RAISE NOTICE 'Total % employees: %', p_gender, p_count;  
END;  
$;
```

Calling the procedure

```
CALL pr_count_by_gender('Male', NULL);  
CALL pr_count_by_gender('Female', NULL);
```

Output

```
NOTICE: Total Male employees: 4  
NOTICE: Total Female employees: 3
```

Result of CALL pr_count_by_gender('Male', NULL); the OUT parameter p_count is returned as a single-row result.

Question 2: Purchase Processing Procedure (Hard)

An e-commerce company maintains product stock information in the products table and stores completed orders in the sales table. Create a PostgreSQL stored procedure named pr_buy_products that accepts a product name and the required quantity as input parameters and processes the purchase only when sufficient stock is available.

Answer

```
CREATE OR REPLACE PROCEDURE pr_buy_products (  
    IN p_product_name VARCHAR,  
    IN p_quantity INT  
)  
LANGUAGE plpgsql  
AS $$
```

```
DECLARE
    v_product_code VARCHAR(10);
    v_price NUMERIC(10,2);
    v_available INT;
    v_total NUMERIC(10,2);
BEGIN
    SELECT product_code, price, quantity_remaining
    INTO v_product_code, v_price, v_available
    FROM products
    WHERE product_name = p_product_name;

    IF NOT FOUND THEN
        RAISE NOTICE 'Product "%" not found.', p_product_name;
        RETURN;
    END IF;

    IF v_available >= p_quantity THEN
        v_total := v_price * p_quantity;

        INSERT INTO sales (order_date, product_code, quantity_ordered, sale_price)
        VALUES (CURRENT_DATE, v_product_code, p_quantity, v_total);

        UPDATE products
        SET quantity_remaining = quantity_remaining - p_quantity,
            quantity_sold = quantity_sold + p_quantity
        WHERE product_code = v_product_code;

        RAISE NOTICE 'Order placed: % x "%" for a total of %.', p_quantity, p_product_name, v_total;
    ELSE
        RAISE NOTICE 'Insufficient stock for "%": requested %, only % available.', p_product_name, p_quantity, v_available;
    END IF;
END;
$$;
```

Demonstration calls

```
CALL pr_buy_products('Wireless Mouse', 5);
CALL pr_buy_products('USB-C Hub', 3);
CALL pr_buy_products('USB-C Hub', 10);
```

Output

```
NOTICE: Order placed: 5 x "Wireless Mouse" for a total of 3495.00.
NOTICE: Order placed: 3 x "USB-C Hub" for a total of 3897.00.
NOTICE: Insufficient stock for "USB-C Hub": requested 10, only 2 available.
```

products table after the calls above

product_code	product_name	price	qty_remaining	qty_sold
P001	Wireless Mouse	699.00	45	5
P002	Mechanical Keyboard	2499.00	20	0
P003	USB-C Hub	1299.00	2	3
P004	Webcam 1080p	1899.00	15	0

sales table after the calls above

order_id	order_date	product_code	qty_ordered	sale_price
1	2026-09-14	P001	5	3495.00
2	2026-09-14	P003	3	3897.00